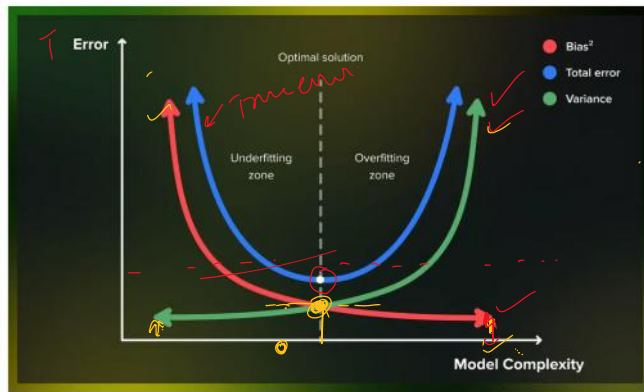




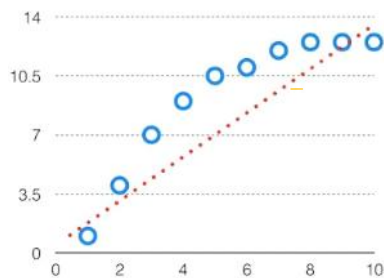
In machine learning, creating models that generalize well to unseen data is essential. Two primary challenges in achieving this are **bias** and **variance**. The key to building an effective model is finding the right balance between these two, a concept known as the **bias-variance trade-off**. Additionally, techniques like **regularization** help control overfitting and improve model performance.

What is Bias?

Bias refers to the error introduced by approximating a real-world problem with a simplified model. A high-bias model makes strong assumptions about the data and may fail to capture its complexity, leading to **underfitting**.

Characteristics of High-Bias Models:

By high bias, the data predicted is in a straight line format, thus not fitting accurately in the data in the data set. Such fitting is known as the **Underfitting of Data**. This happens when the hypothesis is too simple or linear in nature.

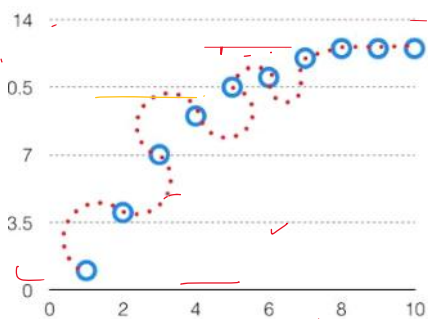


What is Variance?

Variance measures how much the model's predictions fluctuate for different training datasets. A high-variance model is overly complex and captures noise in the training data, leading to **overfitting**.

Characteristics of High-Variance Models:

- Complex models that learn noise along with the patterns.
- Excellent performance on training data but poor generalization to test data.
- Example: Deep decision trees that split on almost every data point.



Finding the Right Balance

The goal is to build a model that achieves low bias and low variance simultaneously. The ideal model is complex enough to capture the underlying patterns in the data while being simple enough to avoid overfitting. This balance is called the **bias-variance trade-off**.

A general guideline:

- If the model has **high bias**, use a more complex model or increase the number of features.
- If the model has **high variance**, apply regularization or gather more training data (more data reduces the effect of noise)

Regularization: Controlling Overfitting

Regularization techniques help prevent overfitting by adding a penalty to large coefficients in a model. The most commonly used regularization methods are **L1 (Lasso)**, **L2 (Ridge)**, and **Elastic Net regularization**.

Bias Variance Trade-off

If the algorithm is too simple (hypothesis with linear equation) then it may be on high bias and low variance condition and thus is error-prone.

high bias and low variance - It makes consistent but wrong predictions.

Low variance means the model's predictions do not change much when the training data changes.
Even if you train the model on different datasets, it still produces similar predictions.

If algorithms fit too complex (hypothesis with high degree equation) then it may be on high variance and low bias. In the latter condition, the new entries will not perform well. There is something between both of these conditions, known as a Trade-off or Bias Variance Trade-off. An algorithm can't be more complex and less complex at the same time.

We try to optimize the value of the total error for the model by using the Bias-Variance Trade-off.

$$\text{TotalError} = \text{Bias}^2 + \text{Variance} + \text{IrreducibleError}$$

The best fit will be given by the hypothesis on the trade-off point. Bias is defined as the **difference between the expected prediction and the true value**.

Cross-Validation

is a resampling procedure used to evaluate machine learning models on a limited data sample. It provides insights into how a model might perform on unseen data, which is crucial for building predictive models that generalize well in real-world applications.

Instead of splitting the data into one training and one testing set, cross-validation divides the dataset into multiple folds (partitions). The model is trained on a subset of the data and validated on the remaining portion. This process is repeated several times, and the performance metrics are averaged to give a more accurate assessment of the model. In essence, cross-validation mimics the process of testing the model on multiple test sets. It helps avoid the risk of the model being overly optimistic (or pessimistic) based on a single data split.

Cross-validation addresses a fundamental challenge in **machine learning**: overfitting. Overfitting occurs when a model performs exceptionally well on training data but poorly on unseen data. This typically happens when the model has memorized the training data rather than learning the underlying patterns.

Cross-Validation repeatedly tests the model on **different unseen parts of the dataset**.

So if the model is memorizing the training data, cross-validation will show:

- **high training accuracy**
- **lower validation accuracy**

This reveals **overfitting early**.

Helps in Hyperparameter tuning:

Suppose you want to tune a hyperparameter like:

- max depth in a decision tree
- regularization strength in regression

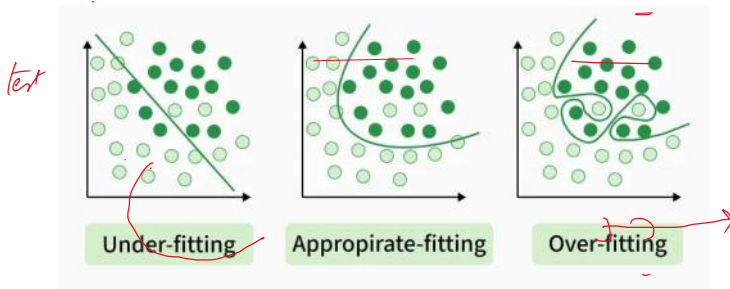
If you only use **one train-test split**, the chosen hyperparameter might perform well **just because of that particular split**, not because it is truly the best.

The hyperparameter with the **highest average score** is selected.

Regularization

Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

- **Prevents overfitting**: Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization**: Encourages simpler models that perform better on new, unseen data.



Types of Regularization

There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.

1. Lasso Regression

A regression model which uses the L1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value

2. Ridge Regression

A regression model that uses the L2 regularization technique is called Ridge regression. It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Lambda: Regularization parameter that controls the strength of regularization

Overfitting often happens when the model learns **very large coefficients** to perfectly fit training data.
Example model:

$$y = 15x_1 - 20x_2 + 12x_3$$

Large coefficients mean:

- Small change in $x \rightarrow$ large change in prediction
- Model becomes **very sensitive to noise**